Apprendre à Cuisto Bongo à jouer tout seul

Rapport de projet — l'apprentissage par renforcement appliqué à Cuisto Bongo, notre jeu de la GameJam 4

Team Bamboo — Tristan Sauvage & Rémi Semenzin

Novembre 2026

Résumé

Cuisto Bongo est un jeu d'action et de cuisine en Python/Pygame réalisé pendant la GameJam 4 : Bongo doit nourrir l'entité de ses prothèses en cuisinant des pizzas, récoltées dans une dimension parallèle, tout en survivant à des vagues d'ennemis de plus en plus nombreuses. Ce rapport décrit la seconde vie du projet : entraîner une intelligence artificielle qui joue au vrai jeu, en ne lui donnant ni règle de décision, ni démonstration.

Le jeu est bien plus complexe que notre précédent projet (Jackass-Parrow) : deux dimensions, une chaîne de production à cinq maillons (traverser un portail, casser une ressource, ramasser l'ingrédient, tuer des tomates, revenir cuisiner), des commandes à livrer dans un temps limité, une boutique d'améliorations, une roulette de malus et de bonus, et un objectif de pizzas qui décide de la fin de partie. Pour que l'IA puisse apprendre, il fallait d'abord pouvoir faire tourner le jeu très vite et sans affichage.

Nous avons donc réécrit les règles du jeu en trois étages : un moteur **headless** en Python pur qui rejoue le vrai jeu à l'identique (plus de 300 000 ticks comparés au jeu Pygame, même graine, mêmes touches, état complet vérifié à chaque tick), puis le même moteur en **tenseurs PyTorch** pour simuler 32 768 parties à la fois sur un GPU (parité tick par tick avec le moteur de référence, aléatoire injecté, bugs volontaires détectés), et enfin un **PPO à actions masquées** entièrement sur le GPU : 57 000 décisions par seconde sur une NVIDIA A100.

L'agent final, entraîné sur **1,45 milliard de décisions en environ 9 heures pour ~16 $** (l'équivalent de 4,6 années de jeu en continu), atteint sur 40 parties jamais vues : **18 321 points en moyenne, 13 vagues validées et 253 pizzas par partie de 21 minutes**, et il est **encore vivant à la coupure dans 37 parties sur 40**. Le hasard fait −417 points et meurt à la première vague ; un bot écrit à la main fait 621 points et 1,6 vague. Le meilleur score humain honnête du scoreboard du jeu est de 2 294 points (vague 4) : l'IA fait huit fois mieux.

Le chemin n'a pas été direct. Le premier agent a appris à survivre parfaitement… sans jamais cuisiner (score −437). La chaîne de production a émergé entre 100 et 200 millions de décisions, puis les gains se sont cumulés vague après vague. Ce rapport détaille la méthode, la vérification de fidélité des moteurs, la chronologie de l'apprentissage et ce que l'agent fait aujourd'hui.

Sommaire

1. Introduction
2. Le jeu Cuisto Bongo
3. Un moteur de simulation sans affichage
4. Formulation du problème d'apprentissage
5. Le moteur vectorisé sur GPU
6. Chronologie de l'entraînement et observations
7. Résultats
8. Analyse du comportement
9. Limites et validité
10. Conclusion et perspectives

Annexe A — Reproduire nos résultats

Annexe B — Glossaire

Annexe C — Références

1. Introduction

1.1 Contexte

Le jeu a été développé en équipe pour la GameJam 4 de l'IUT de Reims. Après le projet Jackass-Parrow (GameJam 3), dont l'IA avait appris à traverser les tableaux et battre le boss, nous voulions appliquer la même méthode à un jeu bien plus riche : Cuisto Bongo mélange un « survivors-like » (hordes, vagues, améliorations) et de la gestion de cuisine (recettes, ingrédients, commandes), avec une dimension parallèle à traverser par des portails. Comme pour Jackass-Parrow, l'entraînement relève de l'**apprentissage par renforcement** (RL) : un agent observe l'état du jeu, choisit une action, reçoit une récompense, et modifie sa **politique** pour maximiser la somme des récompenses futures.

1.2 Objectifs et contraintes

- **Jouer au vrai jeu** : mêmes règles, mêmes vagues, malus, bonus, boutique et roulette compris.
- **Aucune règle de décision codée à la main** : l'agent ne reçoit ni script, ni démonstration. Un bot scripté existe, mais uniquement comme point de comparaison.
- **Résultats vérifiables** : comparer chaque modèle au hasard et au bot scripté sur des graines jamais vues pendant l'entraînement.
- **Pouvoir regarder jouer l'IA** dans les graphismes d'origine, sur un PC sans GPU.

1.3 Organisation du rapport

La partie 2 présente le jeu. La partie 3 décrit le moteur headless et sa vérification contre le jeu réel. La partie 4 formalise le problème (rythme, observation, actions, récompense, algorithme).

La partie 5 décrit le moteur GPU et les deux aides d'entraînement. La partie 6 retrace l'entraînement et ce que nous y avons observé. Les parties 7 et 8 présentent les résultats et l'analyse du comportement. La partie 9 discute les limites ; la partie 10 conclut.

2. Le jeu Cuisto Bongo

2.1 Principe

Ancien cuistot du pirate Jackass-Parrow, Cuisto Bongo porte des prothèses mécaniques habitées par une entité qui doit être constamment rassasiée. Armé de son bongo, de son saxophone et de sa flûte de pan, il survit sur une terre envahie par une dimension parallèle (l'« Upside Down », ciel rose) en faisant des allers-retours entre les portails dimensionnels et son fourneau, pour concocter assez de pizzas.

La boucle centrale est une **chaîne de production** : les ressources (arbres à pâte, cristaux de fromage) ne sont cassables que dans l'Upside Down ; les tomates nécessaires à la sauce sont des ennemis du monde normal ; le four, lui, est dans le monde normal (sauf malus). Il faut donc traverser, récolter, tuer, revenir, cuisiner — le sac à dos étant limité à 3 ingrédients de chaque type au départ.

2.2 Commandes et mécaniques

Tableau 1 — Commandes du joueur (constantes dans `settings.py`).

Commande	Effet
Flèches / ZQSD	Déplacement à 400 px/s dans 8 directions (normalisées en diagonale)
E / Espace	**Interaction universelle** : jouer de l'instrument, ouvrir le menu du four près du four, naviguer dans les menus

Tableau 2 — Les trois instruments (donnés par la roulette de fin de vague, jamais deux fois la même d'affilée).

Instrument	Effet	Recharge
Bongo	Onde de choc autour du joueur (rayon 200) : 10 dégâts aux ennemis, casse les ressources à portée	0,4 s
Saxophone	Note explosive lancée dans la direction du regard : 35 dégâts aux ennemis sur un rayon de 180 après 0,6 s de vol	1,0 s
Flûte de pan	Dash dans la direction du regard (1 500 px/s pendant 0,2 s, invulnérable) : 20 dégâts aux ennemis traversés, casse les ressources sur le passage	1,0 s

Le joueur a 50 PV, 1 s d'invulnérabilité après un coup, et reçoit 5 dégâts au contact d'un ennemi, 10 par projectile de calamar.

Tableau 3 — Les adversaires.

Ennemi	Comportement	PV
Tomate	Patrouille par sauts ; si le joueur est à moins de 220 px, se prépare 1 s puis charge à 320 px/s pendant 1,5 s	30
Calamar	Garde ses distances (150–450 px), tire un projectile visé (350 px/s) toutes les 2 s si le joueur est à moins de 500 px	50
Calamar géant	Comme le calamar, mais 3× plus gros et ses projectiles géants	225

Les ressources (arbres, cristaux) ont 4 et 3 PV et ne sont cassables que dans l'Upside Down ; elles lâchent leur ingrédient au sol. Les ennemis ne peuvent pas entrer dans la zone de sécurité du four (rayon 270), qui détruit aussi les projectiles.

2.3 Structure d'une partie

Une **vague** dure 105 s. Il faut livrer au four un objectif de pizzas (3 à la vague 1, puis +2 par vague jusqu'à 10 ; au-delà, +1 ennemi supplémentaire par vague). Les **commandes** s'affichent par 3 au four : une recette (Margherita 100 pts, Tomate 100, Double Fromage 80, Vide 50), un temps limité (30–50 s) et des points qui fondent avec le temps ; une commande expirée **retire** ses points du score. Si l'objectif de pizzas n'est pas atteint à la fin de la vague : **game over**.

Sinon : **boutique** (dépenser les pizzas cuisinées en améliorations — vitesse, sac, cadence d'attaque, vie, temps de commande ; on ne peut partir que quand plus rien n'est abordable), puis **roulette** d'un malus, d'une arme et d'un bonus (jamais identiques aux précédents). Six malus (invasion de calamars, four dans l'Upside Down, calamars géants, tomates déplacées dans l'Upside Down, objets qui fuient, portails mobiles) et trois bonus (jusqu'à 8 bombes à ramasser, boussole, pilules de vitesse ×1,5 et cadence ×2 pendant 15 s).

La **mort** coûte le contenu du sac, renvoie au spawn et **brûle 30 s sur le chrono de la vague**. La carte fait 5120 × 3200 px ; 4 portails tournants permettent la traversée des dimensions.

Une partie s'arrête au game over — jamais avant.

3. Un moteur de simulation sans affichage

3.1 Pourquoi

Un agent de RL apprend sur des milliards de décisions. Le jeu Pygame tourne à 60 images par seconde avec fenêtre, sons et menus. Il fallait une version de la logique du jeu **rapide, déterministe et indépendante de l'affichage**, sans en changer les règles. À la différence de

Jackass-Parrow, où ce moteur avait d'abord servi à un entraînement sur CPU trop lent, nous avons cette fois visé directement le GPU — mais le moteur headless reste la pièce centrale : c'est la **référence** qui définit les règles, celle qui est comparée au vrai jeu, et celle sur laquelle tous les résultats finaux sont mesurés.

3.2 Choix de conception

- **Bibliothèque standard uniquement** (`rl/headless`) : ni Pygame, ni NumPy. Le moteur tourne partout, y compris dans les processus de la banque de cartes (voir 5.1).
- **Temps discret** : la simulation avance par ticks de 1/60 s ; tous les délais sont comptés en ticks.
- **Aléatoire maîtrisé et fidèle** : chaque tirage du gameplay vient du même générateur dans le même ordre que le jeu. `Engine(seed)` rejoue **exactement** la partie qu'aurait jouée le jeu graphique après `random.seed(seed)` — à condition que les modules purement cosmétiques (secousse de caméra, tuiles de la carte, particules, sons, débris de mort) ne consomment pas le même flux ; le test leur donne leur propre générateur.
- **Formules flottantes identiques** : pygame utilise `sqrt(x*x+y*y)` pour les normes, `a*(1-t)+b*t` pour les interpolations, et ses rectangles tronquent les positions flottantes vers zéro. Le moteur reproduit tout cela à l'identité bit à bit.

Tableau 4 — Abstractions assumées (documentées dans `doc/rl.md`, chacune vérifiée pour ne pas changer les règles).

Dans le jeu	Dans le moteur RL	Justification
Menu des recettes : ouvrir avec E, naviguer, valider	Action « cuisiner la commande k » (four actif + ingrédients présents) ; le menu reste ouvert jusqu'à la décision suivante, le joueur est immobile comme dans le jeu	Même effet, sans apprendre une interface
E près du four ouvre le menu	« Interagir » près du four ne fait rien	Dans le jeu, on ne peut pas attaquer au four
Boutique : naviguer, acheter, « PASSER »	Décisions « acheter la carte k », sans durée ; sortie automatique quand rien n'est abordable	Le jeu impose d'acheter tant qu'une carte est abordable
Animation de mort (7 s), roulette (~6 s)	Instantanées, résultat exact	Le jeu est figé pendant ces phases

3.3 Performances et vérification

`python -m rl.parity` lance **le vrai jeu Pygame** (pilote vidéo factice) et le moteur headless côte à côte, avec la même graine ; un bot envoie de **vraies touches** au jeu (les mêmes touches qu'un joueur, menus compris), et **tout l'état des deux moteurs est comparé à chaque tick** : joueur, 200 ressources, ennemis, projectiles, drops, bombes, pilules, portails, commandes, boutique, roulette, score, tous les compteurs.

Résultat : 307 627 ticks identiques sur 50 graines (jusqu'à 20 000 décisions par graine), couvrant :

Mécanique	Occurrences observées
Les 3 armes (bongo, flûte, saxo)	165 303 / 64 865 / 77 459 ticks d'usage
Les 6 malus et les 3 bonus	tous vus, plusieurs fois chacun
Explosions de bombes / pilules prises	15 / 35
Morts	198
Pizzas livrées / vagues validées / achats	382 / 56 / 139
Notes de saxo explosées / dashs	1 104 / 9 224 ticks
Ticks dans l'Upside Down	144 504

La table de la roulette est vérifiée séparément contre la vraie animation (`ui.RouletteSlot`), frame par frame. `tests/test_headless_parity.py` rejoue une partie de ces graines à chaque `pytest`.

4. Formulation du problème d'apprentissage

4.1 Environnement et rythme de décision

L'agent décide **10 fois par seconde** (une action maintenue 6 ticks, `frame_skip=6`). Une vague de 105 s représente donc 1 050 décisions ; une partie de 21 minutes, 12 600. Une mort n'interrompt pas une décision : l'animation est instantanée dans le moteur.

4.2 Ce que voit l'agent

Une observation numérique **égocentrique** de **577 valeurs** (`CuistoBongo-v1`, `rl/vector/features.py`) : positions relatives au joueur (÷1280, ÷800), entités triées par distance avec indicateur de présence, comme pour Jackass-Parrow. C'est une observation **privilégiée** (positions et délais exacts), pas la vision d'un joueur.

Tableau 5 — Composition de l'observation.

Segment	Taille	Contenu
Joueur	24	position sur la carte, PV, invulnérabilité, dash, direction, vitesse, recharge, sac (contenu, plein), arme
Contexte	35	phase, chrono de vague, vague, objectif, pizzas faites/restantes, cadence nécessaire, dimension, four, malus, bonus, pilule
Four	5	position relative, même dimension, à portée
Commandes	3 × 13	recette, temps restant, ingrédients manquants, cuisinable, points actuels
Boutique	26	3 cartes (amélioration, prix, abordable) + niveau des 5 améliorations
Portails	4 × 7	position, distance, mobile, vitesse
Ennemis proches (même dimension)	8 × 19	position, type (tomate / calamar / géant), PV, état et vitesse des tomates, délais de tir
Tomates (toutes dimensions)	6 × 6	cibles pour la sauce

Projectiles	8 × 7	position, vitesse, taille, âge
Ressources	2 × 6 × 5	arbres (pâte) et cristaux (fromage) les plus proches, PV
Ingrédients au sol	8 × 9	type, ramassable dans la dimension courante, sac plein pour ce type
Bombes, pilules, notes de saxo	24 + 12 + 8	positions, dimension, mèche

4.3 Ce que peut faire l'agent

24 actions discrètes, masquées selon l'état (`rl/actions.py`) :

Index	Action	Valide quand
0–8	se déplacer (immobile + 8 directions)	en jeu
9–17	se déplacer + jouer de l'instrument	en jeu
18–20	cuisiner la commande 1, 2, 3	four actif, ingrédients présents
21–23	acheter la carte 1, 2, 3	boutique, carte abordable

Le **masque** écarte de la distribution les actions impossibles : l'agent n'explore pas des milliers d'appuis sur E sans effet, et la probabilité des actions invalides est exactement nulle (logit à -10^8).

4.4 La récompense

Le score du jeu seul est un mauvais signal : il ignore l'objectif de pizzas qui met fin à la partie. Nous avons défini une récompense d'apprentissage comptée par le moteur (`rl/rewards.py`) :

Tableau 6 — La récompense d'apprentissage. Aucune récompense pour survivre sans cuisiner.

Événement	Poids	Raison
Points de commande (une expirée retire ses points)	× 0,01	le score du jeu
Pizza livrée	+1	c'est l'objectif de pizzas qui décide de la survie
Vague validée	+3	
Game over (objectif raté)	−5	fin de partie
Mort	−1	sac vidé, 30 s perdues
PV perdu	−0,01	

Deux principes hérités de Jackass-Parrow : ne jamais récompenser la survie en elle-même (un agent payé pour durer apprend à ne rien faire, nous l'avons revérifié — voir 6.1), et récompenser les événements du jeu, pas les intentions.

4.5 L'algorithme : PPO à actions masquées

PPO (Schulman et al., 2017) : acteur-critique, avantage par GAE, mise à jour bornée (clip 0,2). Le choix, mûri pendant le projet Jackass-Parrow, se justifie ainsi pour Cuisto Bongo :

- la simulation tourne sur un seul GPU avec 32 768 parties en parallèle : c'est le terrain des méthodes **on-policy** à gros lots, et PPO est la plus robuste d'entre elles ;

- les alternatives n'apportent rien ici : DQN/Rainbow et SAC-discret ont besoin d'une mémoire de rejeu (inutile quand on génère 57 000 décisions/s fraîches) et profitent mal du GPU pour la collecte ; IMPALA sert à répartir des acteurs sur plusieurs machines ; MuZero/Dreamer apprennent un modèle du monde alors que nous avons **le simulateur exact** ;
- les **actions masquées** (une seule tête catégorielle) gèrent proprement les trois contextes (jeu, four, boutique) sans tête ni époque spéciale ;
- pas de mémoire récurrente : l'observation contient tous les délais (recharge, invulnérabilité, mèche de bombe, chrono de commande), l'état est markovien.

Réseau : acteur et critique séparés, MLP 512-512-256, activation SiLU, normalisation courante des observations sauvegardée dans le checkpoint. Un encodeur d'entités par attention serait plus général, mais sur 577 valeurs triées, un MLP suffit et coûte peu face à la simulation — c'est la première piste d'amélioration si l'agent plafonne.

Tableau 7 — Hyperparamètres de l'entraînement.

Hyperparamètre	**Valeur**		**Hyperparamètre**	**Valeur**
γ (actualisation)	0,998		Époques / mini-lots	4 / 8
λ (GAE)	0,95		Coefficient d'entropie	0,01
Clip PPO	0,2		Coefficient de valeur	0,5
Taux d'apprentissage	$3 \cdot 10^{-4}$ → 10 %		Norme max. du gradient	0,5
Parties parallèles	32 768		Pas par partie et par mise à jour	16
Décisions par mise à jour	524 288		Graine	1
Partie d'entraînement	12 000 décisions (20 min)		Précision de simulation	float32

γ est plus élevé que pour Jackass-Parrow (0,995 à 15 décisions/s) : un cycle « portail → récolte → retour → four » dure 20 à 40 s, et l'horizon effectif (≈ 50 s) doit couvrir la chaîne.

5. Le moteur vectorisé sur GPU

5.1 Principe

`rl/vector/engine.py` réécrit toutes les règles en opérations sur des **tenseurs PyTorch** : chaque variable d'état est un tableau de N parties (jusqu'à 32 768). Les listes deviennent des emplacements fixes avec masque : 205 ressources, 75 ennemis, 243 drops, 48 projectiles, 8 bombes, 8 pilules, 4 notes, 3 commandes. L'ordre des mises à jour est celui du moteur de référence.

- **L'ordre séquentiel des ennemis est conservé** : dans le jeu, chaque ennemi voit les nouvelles positions des précédents (séparation de Gauss-Seidel). C'est une boucle sur les emplacements d'ennemis — le seul endroit non vectorisable sans trahir les règles.
- **Banque de cartes** : faire apparaître une vague demande un échantillonnage par rejet séquentiel (jusqu'à 50 essais par entité), impossible à vectoriser honnêtement. `rl/vector/layouts.py` génère donc à l'avance **16 384 cartes complètes** (ressources, calamars, tomates, spots d'invasion, de géants, de bombes, de pilules) avec le code de spawn du moteur headless, en cache disque ; chaque début de vague en tire une au hasard.
- **Compatible graphe CUDA** : aucun synchronisation GPU → CPU dans un pas (pas de `.item()`, pas de forme dépendante des données) ; le pas complet — simulation, observation, récompense, remise à zéro — est capturé dans un **graphe CUDA** et rejoué en un seul appel.

5.2 La fidélité se teste

Un moteur réécrit peut différer subtilement du jeu. `tests/test_vector_engine.py` (13 tests) copie à chaque tick l'état du moteur headless dans le moteur torch, fait avancer les deux **avec les mêmes nombres aléatoires injectés**, et compare tout l'état (float64, tolérance 10^{-9}) : 9 894 ticks comparés sur 11 parties, couvrant les 3 armes, les 6 malus, les 3 bonus (dont deux bombes explosant ensemble, pilules, calamars géants, objets qui fuient, portails mobiles, four dans l'Upside Down), les morts, la cuisson menu ouvert, les changements de dimension, la boutique, la roulette déclenchée dans un tick, et le game over.

Pour vérifier que le test détecte vraiment les écarts, six bugs volontaires ont été injectés — **tous détectés** :

Bug injecté	**Détecté à**
Rayon de séparation des calamars 80 → 81	tick 2
Invulnérabilité après un coup 1 → 0,9 s	tick 56
Séparation sur les anciennes positions de tous les ennemis	tick 0
Capacité du sac décalée de 1	tick 0
Force de séparation des tomates +1	tick 0
Rayon des bombes 300 → 250	tick 272

Tous les résultats finaux sont mesurés sur le **moteur headless**, pas sur la version GPU.

5.3 Débits mesurés

Tableau 8 — Débits mesurés le 6 novembre 2026.

Configuration	**Décisions/s**
Moteur torch, CPU local (Ryzen AI 9 365), 1 024 parties, float64 / float32	3 613 / 3 949
A100-SXM4-80GB, 16 384 parties, sans graphe CUDA, float64 / float32	43 139 / 45 698
A100, 32 768 parties, graphe CUDA, entraînement complet	**≈ 57 000**

Le plafond vient de la boucle séquentielle des ennemis : 75 emplacements × 6 ticks = 450 itérations par décision, chacune une chaîne de dépendances de petits noyaux. C'est une limite de **latence**, pas de bande passante : passer de 16 384 à 65 536 parties ne change presque rien (55 000 → 59 000 en simulation seule). La leçon de Jackass-Parrow reste vraie : le débit est le verrou, et il vaut mieux un moteur séquentiel-fidèle à 57 000 décisions/s qu'un moteur triché à 500 000.

5.4 PPO entièrement sur le GPU

Le lanceur `rl/train.py` garde tout sur la carte : collecte, avantages, mises à jour. Normalisation des observations dans le réseau (sauvegardée avec le modèle), récompenses divisées par l'écart-type courant du retour (ReturnScaler), acteur et critique séparés, TF32 pour le réseau (pas pour la simulation), taux d'apprentissage décroissant, checkpoints toutes les 50 M de décisions, évaluation périodique de 512 parties et sauvegarde du meilleur.

5.5 Deux aides d'entraînement

Un façonnage de récompense par potentiel. Le geste décisif — récolter ce que la commande demande, puis se tenir près du four quand tout est prêt — est rare au hasard. Nous ajoutons $F = \gamma \cdot \Phi(s') - \Phi(s)$, avec $\Phi = 0{,}25$ par ingrédient du sac encore demandé par une commande ouverte + 0,5·(1 − distance au four / 4 000 px) quand une commande est cuisinable (distance via le meilleur portail si le four est dans l'autre dimension). Φ est nul hors jeu et après un état terminal. Cette forme ne change pas la politique optimale (Ng, Harada et Russell, 1999) : ce qu'on gagne en s'approchant, on le reperd en s'éloignant — pas de bonus à exploiter en tournant en rond.

Des départs variés. La moitié des parties d'entraînement commencent directement à une vague tirée entre 2 et 8, avec l'objectif et les ennemis supplémentaires de cette vague, des améliorations achetées avec les pizzas des vagues précédentes, et une arme, un malus et un bonus tirés au hasard. L'agent rencontre ainsi tous les malus et toutes les armes dès le début, au lieu d'attendre de savoir y arriver. Les statistiques ne comptent que les parties commencées normalement.

6. Chronologie de l'entraînement et observations

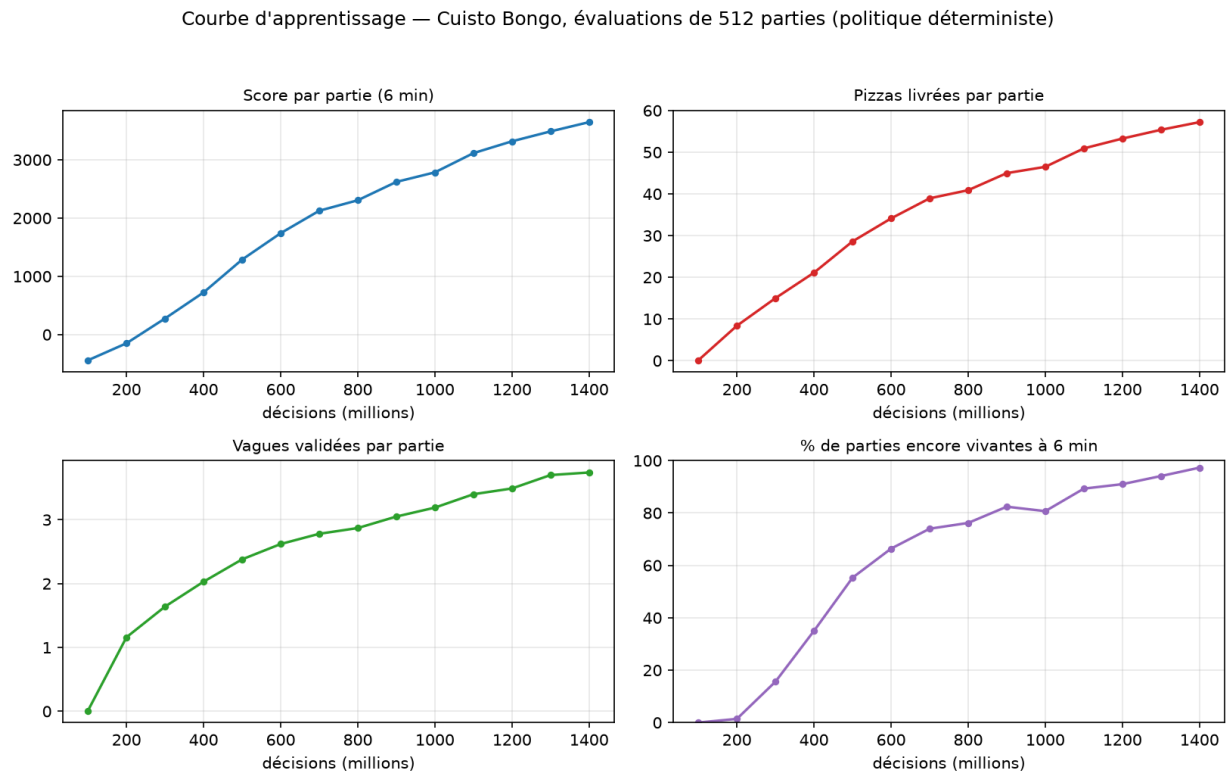
L'entraînement (`bongo-v1`, graine 1) a tourné sur un pod RunPod **NVIDIA A100-SXM4-80GB** (80 Go, 1,79 $/h), du 6 novembre 10 h 32 à 19 h 50, interrompu à 1,45 Md de décisions sur les 2 Md prévus (solde du compte épuisé — voir 9). Ce qui suit est la chronologie des évaluations périodiques de 512 parties (politique déterministe, départs normaux, **fenêtre de 6 minutes**).

Tableau 9 — Les évaluations périodiques.

Décisions	Score	Pizzas	Vagues	Vivantes à 6 min
100 M	−441	0,1	0,01	0 %
200 M	−149	8,4	1,16	1 %
300 M	+277	15,0	1,64	16 %

400 M	+724	21,1	2,03	35 %
500 M	+1 288	28,6	2,38	55 %
600 M	+1 741	34,1	2,62	66 %
700 M	+2 126	38,9	2,78	74 %
800 M	+2 306	40,9	2,87	76 %
900 M	+2 621	45,0	3,05	82 %
1 000 M	+2 785	46,5	3,19	81 %
1 100 M	+3 115	50,9	3,40	89 %
1 200 M	+3 318	53,3	3,49	91 %
1 300 M	+3 488	55,4	3,70	94 %
1 400 M	+3 647	57,2	3,74	97 %

Figure 1 *— Courbe d'apprentissage (`presentation/courbe-apprentissage.png`).*



6.1 0–50 M : l'agent apprend à camper

Le premier checkpoint (50 M), évalué sur 40 graines officielles, donne le tableau le plus instructif du projet : score −437, **0,00 mort par partie**, épisodes d'exactement 105 s — et **0 pizza, 0 récolte**. L'agent a trouvé l'optimum local « survivre sans rien tenter » : il esquive (0,95 ennemi tué par partie), ne meurt jamais, et laisse expirer toutes les commandes. C'est la confirmation directe du principe « ne jamais récompenser la survie » : même sans bonus de survie, la seule récompense accessible (éviter −5 de game over) se gagne en attendant.

Leçon : la survie ou la récompense moyenne ne disent rien de la qualité d'un agent. Seuls le score, les pizzas et les vagues comptent — et il faut le regarder jouer.

6.2 50–100 M : la survie parfaite, toujours 0 pizza

À 100 M, l'évaluation montre la même image (−441, 0,1 pizza, 100 % de game over à la vague 1). Le façonnage par potentiel n'a pas encore produit son effet : ramasser un ingrédient utile rapporte Φ, mais il faut d'abord **découvrir** que casser un arbre dans l'Upside Down fait tomber de la pâte — un maillon invisible de la chaîne.

6.3 100–200 M : la chaîne émerge

Entre 100 M et 200 M, tout se débloque : 8,4 pizzas par partie, la vague 1 validée dans la moitié des parties. Une fois le premier maillon trouvé (récolter), le reste de la chaîne suit vite : les gradients du potentiel guident le sac (ramasser ce que la commande demande), puis le retour au four (bonus de proximité quand tout est prêt). C'est exactement le rôle prévu du façonnage : rendre la séquence complète visible, sans changer la politique optimale.

6.4 200 M–1,45 Md : les vagues s'enchaînent, la pente s'adoucit

De 200 M à 500 M, le score passe de −149 à +1 288 et l'agent dépasse le bot scripté sur toutes les mesures entre 300 M et 400 M. Ensuite, les gains par palier de 100 M décroissent régulièrement (+447 entre 400 et 500 M, +330, +170, +159…). Deux causes lisibles dans les chiffres :

- la **fenêtre d'évaluation de 6 min sature** : 105 s par vague, c'est 3,4 vagues maximum ; à 1,1 Md, 89 % des parties sont encore vivantes à la coupure et seules les pizzas continuent de différencier les modèles ;
- la compétence restante est la **survie en vagues avancées** : le nombre de morts par partie d'éval monte de 0,7 (300 M) à 3,4 (1,4 Md) pendant que le score grimpe — l'agent pousse plus loin dans les vagues à 10 pizzas et ennemis supplémentaires, où il se fait encore toucher.

En fin d'entraînement, les statistiques des parties d'entraînement (fenêtre de 20 min) montrent l'ampleur : **16 700 points, 12,3 vagues validées et 233 pizzas par partie** à 1,43 Md.

6.5 Un incident, aucun progrès perdu

À 19 h 50, le solde RunPod atteint zéro et le pod est coupé automatiquement à 1,45 Md de décisions (72 % du budget). Tous les checkpoints et les métriques étaient sur le volume persistant du pod ; le run est reproductible à la décision près depuis le checkpoint 1,45 Md (`--resume`). Nous avons choisi de considérer ce point comme le modèle final : la courbe était encore en montée douce, mais le niveau atteint dépassait déjà très largement nos objectifs (voir 7).

7. Résultats

7.1 Protocole d'évaluation

- **Moteur de référence** headless (mêmes règles que le jeu graphique, vérifié tick par tick).

- **40 graines jamais utilisées à l'entraînement** (700 000 à 700 039), identiques pour tous les joueurs comparés.
- **Parties complètes jusqu'au game over**, coupées à 21 minutes (12 600 décisions).
- Politique **déterministe** (l'action la plus probable).

7.2 Comparaison finale

Tableau 10 — Comparaison sur 40 parties inédites (moteur de référence, mêmes graines).

Joueur	Score	Vagues validées	Pizzas	Morts	Game over
Hasard	−417 ± 71	0,0	0,0	0,15	40 / 40
Bot scripté (`rl/baselines.py`)	621 ± 466	1,6	12,4	5,2	40 / 40
Agent `bongo-v1` (1,45 Md)	**18 321 ± 3 663**	**13,1 ± 2,2**	**253 ± 48**	9,1	**3 / 40**

Détail de la distribution de l'agent : médiane 19 349 points ; 37 des 40 parties atteignent la coupure de 21 min **encore vivantes** (vagues 13 à 15 validées) ; les 3 game over tombent à 5,5 min (2 473 pts), 8,9 min (6 052 pts) et 14,6 min (10 876 pts). Pour référence, le meilleur score humain honnête du scoreboard du jeu est **2 294 points (vague 4)** : l'agent fait huit fois mieux et tient trois fois plus de vagues.

7.3 Coûts

L'entraînement a duré **environ 9 heures** sur une A100-SXM4-80GB louée (1,79 $/h), soit **≈ 16 $**, pour 1,45 milliard de décisions — l'équivalent de **4,6 années de jeu en continu** (à 10 décisions/s). Regarder ou évaluer le modèle ne demande qu'un CPU (quelques secondes par partie sur le moteur headless).

8. Analyse du comportement

8.1 Méthode

Les 40 parties d'évaluation de l'agent final sont instrumentées par le moteur de référence (totaux par partie) ; le comportement de détail a été observé dans le **mode spectateur** (`rl/watch.py`), qui fait jouer le modèle au vrai jeu Pygame — fenêtre, sons et graphismes d'origine — en pressant les touches comme un joueur.

8.2 Ce que les compteurs disent

Tableau 11 — Mesures par partie de 21 minutes (40 parties, moteur de référence).

Mesure	Agent 1,45 Md	Hasard	Bot scripté
Pizzas livrées	253	0	12,4
Commandes expirées	1,8	5,25	0,65
Ressources cassées	688	0,95	38,9
Ennemis tués	271	2,9	12,25
Morts	9,1	0,15	5,2
Parties vivantes à 21 min	37 / 40	0 / 40	0 / 40

- **Une pizza toutes les 5 secondes** en moyenne : la chaîne portail → récolte → retour → four est devenue un réflexe, avec 688 ressources cassées et 271 ennemis tués par partie pour alimenter les commandes.
- **Les commandes expirées (1,8/partie)** montrent qu'il gère les priorités : à 3 commandes simultanées, il livre presque tout, contre 5,25 expirées pour le hasard.

8.3 Observations en spectateur

- Il traverse les portails **en ligne droite**, sans hésitation, même quand le malus les fait bouger.
- Il récolte **ce que la commande demande** : la séquence typique est un aller dans l'Upside Down ciblé sur les arbres ou les cristaux les plus proches, remplissage du sac, retour immédiat.
- Au four, il enchaîne les livraisons dès que les ingrédients sont là ; en boutique, il achète sans temps mort (le moteur rend la boutique instantanée, mais ses choix d'améliorations sont cohérents d'une partie à l'autre).
- Il exploite les bonus : les bombes ramassées nettoient des groupes entiers, et sous pilule (vitesse ×1,5) sa cadence de livraison s'envole.
- Il meurt surtout dans les vagues avancées, en récoltant sous la pression d'ennemis supplémentaires — jamais en début de partie.

8.4 Ce qu'il reste à apprendre

Le plafond observable est la **vague 13–15** : objectif de 10 pizzas, ennemis supplémentaires cumulés (+1 par vague au-delà de la vague 5), tous les malus en rotation. Les trois défaites de l'évaluation montrent les situations qui restent difficiles : une partie perdue à la vague 4 en 5,5 min, deux autres aux vagues 7 et 10. La courbe d'entraînement étant encore en montée à l'arrêt, ces situations sont précisément ce que les 550 M de décisions restantes du budget auraient travaillé.

9. Limites et validité

- **Observation privilégiée.** L'agent reçoit positions et délais exacts, pas l'image du jeu. Sa tâche est plus facile que celle d'un humain ; la comparaison au scoreboard humain est indicative, pas une égalité de conditions.
- **Un seul entraînement final** (graine 1), interrompu à 1,45 Md sur 2 Md (solde du compte GPU). La régularité entre graines d'évaluation est bonne (±3 663 points), mais la stabilité de la méthode (relancer avec une autre graine) n'est pas démontrée.
- **Pas d'ablation.** L'effet propre du façonnage, des départs variés, du masque d'actions ou de l'observation a été introduit ensemble ; nous ne pouvons pas attribuer le résultat à chacun. La chronologie (6.1–6.3) suggère fortement que le façonnage a débloqué la chaîne, mais c'est une lecture, pas une mesure isolée.

- **Précision de simulation float32.** La parité exacte du moteur torch se vérifie en float64 (coûteux sur GPU) ; l'entraînement a utilisé float32 (approximation documentée, testée hors tolérance ~10^{-9} sur les champs continus).
- **Approximations du moteur GPU** : la banque de cartes suppose les portails à leur position d'origine au moment du spawn (faux après le malus « portails pompettes ») ; le nombre d'ennemis supplémentaires est plafonné à 12 (vague 17) ; au plus une note de saxo explose par tick. Toutes documentées dans `rl/vector/engine.py`, et tous les résultats finaux sont mesurés sur le moteur headless, qui n'a aucune de ces approximations.
- **Fenêtre d'évaluation saturée** : la mesure « vagues » des évaluations périodiques plafonne à ~3,4 (6 min) ; les chiffres de vagues du tableau 10 viennent de parties complètes.
- **Récompense choisie à la main** : les coefficients sont des hypothèses raisonnées, pas des valeurs optimisées.
- **Le jeu lui-même** : l'entraînement s'arrête à la vague 13–15, mais la difficulté du jeu continue de croître linéairement (ennemis supplémentaires) ; nous ne savons pas où l'agent finirait par tomber sans limite de temps.

10. Conclusion et perspectives

Le projet a abouti à un agent qui joue au vrai Cuisto Bongo : 18 321 points et 13 vagues validées par partie de 21 minutes, 253 pizzas livrées, encore vivant dans 37 parties sur 40 — là où le hasard meurt à la première vague et où un bot écrit à la main s'effondre à la deuxième. Au-delà du résultat, cinq leçons :

11. **La parité d'abord.** Un moteur headless qui rejoue le vrai jeu tick par tick (307 000 ticks vérifiés) et un moteur GPU testé avec l'aléatoire injecté (6 bugs volontaires détectés) : c'est ce qui permet de faire confiance à tout le reste.
12. **Le débit reste le verrou, mais la fidélité d'abord.** 57 000 décisions/s sur A100 — 15× le CPU local — suffisent parce que la boucle séquentielle des ennemis a été conservée au lieu d'être vectorisée « à la Jacobi », ce qui aurait trahi les règles pour un facteur 3.
13. **Le masque d'actions rend l'interface apprenable.** Cuisiner et acheter ne sont choisissables que quand c'est possible : l'agent n'a jamais eu à apprendre à naviguer dans des menus.
14. **Le façonnage par potentiel fait émerger les chaînes longues.** Sans lui, l'agent campe (0 pizza à 100 M) ; avec lui, la chaîne complète apparaît entre 100 et 200 M. Sans changer la politique optimale.
15. **Comparer au hasard ET à un bot scripté.** Le bot a été dépassé entre 300 et 400 M de décisions ; sans ce repère, nous aurions pu arrêter « un agent qui cuisine » bien trop tôt.

Perspectives : finir les 550 M de décisions restants du budget (la courbe montait encore) ; relancer avec plusieurs graines et ablations (façonnage, départs variés, masque) ; un encodeur d'entités par attention pour dépasser le MLP ; entraîner sur des parties plus longues ; et, comme pour Jackass-Parrow, apprendre à partir des pixels pour une IA qui « voit » le jeu comme un joueur.

Annexe A — Reproduire nos résultats

```
# Environnement (Windows, CPU suffit pour regarder et évaluer) :
uv sync --extra train
uv run pytest -q                      # parité jeu/headless/torch (~2 min)

# Regarder le modèle final dans le vrai jeu (fenêtre, sons) :
uv run python -m rl.watch --player runs/pod-checkpoints/step-1.45B.pt --speed 2
#   Espace pause · ↑/↓ vitesse ×1-×8 · N nouvelle partie · Tab déterministe/échantillonné
· Échap quitter

# Évaluer sur les 40 graines de test (moteur de référence, parties complètes) :
uv run python -m rl.evaluate --player runs/pod-checkpoints/step-1.45B.pt --games 40 --seed
700000 --limit 12600
uv run python -m rl.evaluate --player scripted --games 40 --seed 700000 --limit 12600
uv run python -m rl.evaluate --player random --games 40 --seed 700000 --limit 12600

# Vérifier la parité contre le vrai jeu (plus long) :
uv run python -m rl.parity --seeds 1 2 3 --decisions 20000

# Réentraîner sur GPU (RunPod A100, ~9 h pour 1,45 Md de décisions) :
uv run python -m rl.train --output runs/bongo-v1 --envs 32768 --n-steps 16 --timesteps
1450000000 --seed 1
# Reprendre depuis un checkpoint (réseau, optimiseur, normalisations, compteur) :
uv run python -m rl.train --output runs/bongo-v1 --resume runs/bongo-
v1/checkpoints/step_1450180608.pt --timesteps 2000000000
```

Annexe B — Glossaire

Terme	**Définition**
Tick	Pas de simulation physique de 1/60 s
Décision	Choix d'une action par l'agent, maintenue 6 ticks (0,1 s)
Épisode	Une partie, de la réinitialisation au game over (ou à la coupure)
Politique	Fonction qui associe à une observation une probabilité pour chaque action
Acteur-critique	Réseau qui choisit les actions (acteur) + réseau qui estime leur valeur (critique)
PPO	Proximal Policy Optimization : mises à jour de politique bornées pour rester stables
GAE	Generalized Advantage Estimation : estimation de l'avantage d'une action, réglée par λ
Façonnage par potentiel	Bonus $\gamma\Phi(s') - \Phi(s)$ qui guide l'exploration sans changer la politique optimale
Masque d'actions	Les actions impossibles reçoivent une probabilité nulle
Graphe CUDA	Séquence d'opérations GPU enregistrée une fois puis rejouée en un seul appel
Headless	Exécution sans affichage
Banque de cartes	Vagues pré-générées par le code de spawn du moteur headless, tirées au hasard
Graine	Valeur initiale du générateur aléatoire ; même graine = même partie

Annexe C — Références

- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. *Proximal Policy Optimization Algorithms*. 2017.
- A. Y. Ng, D. Harada, S. Russell. *Policy invariance under reward transformations: theory and application to reward shaping*. ICML 1999.
- J. Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2016.
- Rapport du projet précédent : *Apprendre à un pirate à jouer tout seul* (Jackass-Parrow, GameJam 3).
- Documentation du projet : `doc/rl.md`, `README.md`.

Cuisto Bongo — rapport IA · fin